

Terraform-05

1). Watch the **Terraform-05** video.

- Remote backend and state locking
- Create s3 using terraform

```
main.tf > ...
1 resource "aws_s3_bucket" "s3_bucket" {
2     bucket = "terraform-backend-arun-987654"
3
4     acl = "private"
5 }
6
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.50.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\LENOVO\Desktop\Terraform-basic> |
```

- Terraform apply

```
Comments will detect it and remind you to do so if necessary.
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
+ create

Terraform will perform the following actions:

# aws_s3_bucket.s3_bucket will be created
+ resource "aws_s3_bucket" "s3_bucket" {
  + acceleration_status = (known after apply)
  + acl                 = "private"
}
```

- Added bucket

```
Enter a value: yes

aws_s3_bucket.s3_bucket: Creating...
aws_s3_bucket.s3_bucket: Creation complete after 7s [id=terraform-backend-arun-987654]

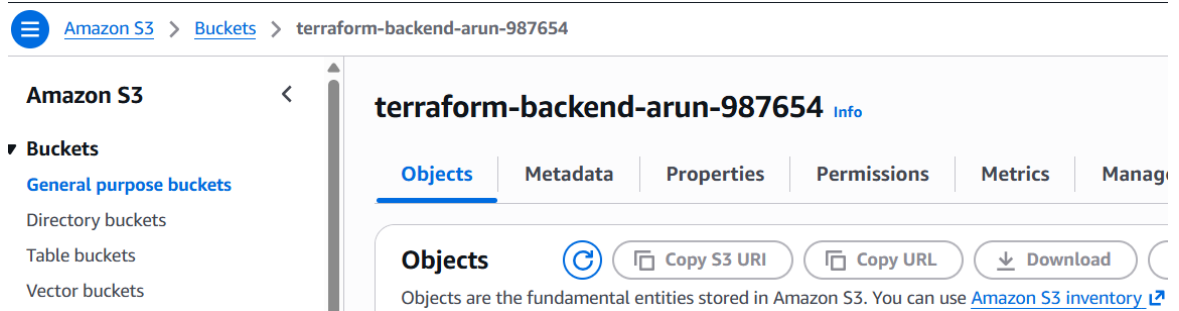
Warning: Argument is deprecated

  with aws_s3_bucket.s3_bucket,
  on main.tf line 4, in resource "aws_s3_bucket" "s3_bucket":
   4:     acl = "private"

acl is deprecated. Use the aws_s3_bucket_acl resource instead.

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

- Successfully bucket created in S3



- Now Create dynamo db using terraform

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "s3_bucket" {
  bucket = "terraform-backend-arun-987654"
}

resource "aws_dynamodb_table" "terraform_state_lock" {
  name             = "terraform-state-lock-dynamo"
  billing_mode     = "PROVISIONED"
  read_capacity    = 20
  write_capacity   = 20

  hash_key = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }
}
```

- Terraform init

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.50.0

Initializing the backend...

Initializing provider plugins found in the state...
- Reusing previous version of hashicorp/aws
- Using previously-installed hashicorp/aws v6.50.0

Terraform has been successfully initialized!
```

- Terraform apply

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
aws_s3_bucket.s3_bucket: Refreshing state... [id=terraform-backend-arun-987654]

Terraform used the selected providers to generate the following execution plan. Resource actions
+ create

Terraform will perform the following actions:

# aws_dynamodb_table.terraform_state_lock will be created
+ resource "aws_dynamodb_table" "terraform_state_lock" {
```

```

Enter a value: yes

aws_dynamodb_table.terraform_state_lock: Creating...
aws_dynamodb_table.terraform_state_lock: Still creating... [00m10s elapsed]
aws_dynamodb_table.terraform_state_lock: Creation complete after 10s [id=terraform-state-lock-dynamo]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

```

- Now check in DynamoDB created

The screenshot shows the Amazon DynamoDB console. On the left is a navigation menu with options like Dashboard, Tables, Explore items, PartiQL editor, Backups, Exports to S3, Imports from S3, and Integrations. The main area displays 'Tables (1)' with a search bar and a filter by tag dropdown. A table lists the created table:

Name	Status	Partition key
terraform-state-lock-dynamo	Active	LockID (S)

- S3 as backend for terraform.tfstate file

The screenshot shows a code editor with two tabs: 'main.tf' and 'terraform.tfstate'. The 'terraform.tfstate' tab is active, showing the following configuration:

```

21 terraform {
22   backend "s3" {
23     bucket = "terraform-backend-arun-987654"
24     dynamodb_table = "terraform-state-lock-dynamo"
25     key = "terraform.tfstate"
26     region = "us-east-1"
27   }
28 }

```

Below the code editor is a terminal window showing the output of the 'terraform init' command:

```

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.50.0

```

- In S3 bucket terraform.tfstate file created

The screenshot shows the Amazon S3 console. The left navigation pane shows 'Amazon S3' with options like Buckets, Files, and Access management and security. The main area displays the bucket 'terraform-backend-arun-987654'. The 'Objects' tab is selected, showing a list of objects:

Name	Type	Last modified
terraform.tfstate	tfstate	June 12, 2026, 19:17:34 (UTC+05:30)

- In DynamoDB also created

Filters - optional

Run Reset

Completed · Items returned: 1 · Items scanned: 1 · Efficiency: 100% · RCUs consumed: 0.5

Table: terraform-state-lock-dynamo - Items returned (1)

Scan started on June 12, 2026, 19:25:39

LockID (String)	Digest
terraform-backend-ar...	cc651c82815034dbfc69021930835f8d

2). Execute the script shown in the video.

- Terraform Provisioner:

```
main.tf > resource "aws_instance" "webserver" > abc key_name
1 resource "aws_instance" "webserver" {
2   ami = "ami-0521cb2d60cfbb1a6"
3   instance_type = "t3.micro"
4   key_name = "aws"
5
6   tags = {
7     NAME = "First-server"
8   }
9 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.50.0...
- Installed hashicorp/aws v6.50.0 (signed by HashiCorp)
```

- Terraform apply

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
+ create

Terraform will perform the following actions:

# aws_instance.webserver will be created
+ resource "aws_instance" "webserver" {
+   ami              = "ami-0521cb2d60cfbb1a6"
+   arn              = (known after apply)
+   associate_public_ip_address = (known after apply)
+   availability_zone = (known after apply)
```

```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.webserver: Creating...
aws_instance.webserver: Still creating... [00m10s elapsed]
aws_instance.webserver: Creation complete after 17s [id=i-02ab2a31443beb4b5]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

```

- New EC2 instance created

Instances (1) [Info](#) Last updated less than a minute ago [Refresh](#) [Connect](#) [Instance state](#)

Saved filter sets: Running

Instance state = running

☐	Name	Instance ID	Instance state	Instance type	Status check
☐		i-02ab2a31443beb4b5	Running v v	t3.micro	Initializing v

- Local Provisioner:

```

resource "aws_instance" "webserver" {
  ami           = "ami-0521cb2d60cfbb1a6"
  instance_type = "t3.micro"
  key_name      = "aws"

  tags = {
    Name = "First-server"
  }

  provisioner "local-exec" {
    command = "echo EC2 instance created successfully > instance_state.txt"
  }
}

```

- Terraform init

```

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.50.0

Initializing the backend...

Initializing provider plugins found in the state...

Terraform has been successfully initialized!

```

➤ Terraform apply

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply

Terraform used the selected providers to generate the following execution plan.
+ create

Terraform will perform the following actions:

# aws_instance.webserver will be created
+ resource "aws_instance" "webserver" {
  + ami           = "ami-0521cb2d60cfbb1a6"
  + arn           = (known after apply)
  + associate_public_ip_address = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.webserver: Creating...
aws_instance.webserver: Still creating... [00m10s elapsed]
aws_instance.webserver: Provisioning with 'local-exec'...
aws_instance.webserver (local-exec): Executing: ["cmd" "/C" "echo EC2"]
aws_instance.webserver: Creation complete after 17s [id=i-0735c52dd3b29a35e]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

➤ Instance has been created

Instances (1) [Info](#) Last updated less than a minute ago [Connect](#) [Instance state](#)

Saved filter sets
[Choose filter set](#)

[Instance state = running](#) [Clear filters](#)

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability zone
☐	First-server	i-0735c52dd3b29a35e	Running	t3.micro	3/3 checks passed	us-east-1

• Now when I use destroy

```
main.tf > resource "aws_instance" "webserver" > provisioner "local-exec"
1  resource "aws_instance" "webserver" {
6  tags = {
7      Name = "First-server"
8  }
9
10 provisioner "local-exec" {
11     when      = destroy
12     command = "echo Instance ${self.public_ip} destroyed! > instance_state.txt"
13 }
14 }
```

• Terraform destroy it will automate terminate instance

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform destroy

aws_instance.webserver: Refreshing state... [id=i-0735c52dd3b29a35e]

Terraform used the selected providers to generate the following execution plan.
- destroy

Terraform will perform the following actions:

# aws_instance.webserver will be destroyed
```

```

Enter a value: yes

aws_instance.webserver: Destroying... [id=i-0735c52dd3b29a35e]
aws_instance.webserver: Provisioning with 'local-exec'...
aws_instance.webserver (local-exec): Executing: ["cmd" "/C" "echo Instance 3.87.80.33 destr
aws_instance.webserver: Still destroying... [id=i-0735c52dd3b29a35e, 00m10s elapsed]
aws_instance.webserver: Still destroying... [id=i-0735c52dd3b29a35e, 00m20s elapsed]
aws_instance.webserver: Still destroying... [id=i-0735c52dd3b29a35e, 00m30s elapsed]
aws_instance.webserver: Destruction complete after 32s

Destroy complete! Resources: 1 destroyed.

```

- Check in AWS instance terminated

Instances (1/3) [Info](#) Last updated less than a minute ago 

Saved filter sets
Terminated 

Instance state = terminated  [Clear filters](#) 

	Name 	Instance ID	Instance state 	Instance type
☐		i-02ab2a31443beb4b5	 Terminated  	t3.micro
☐		i-0d29dc1ade05a0d9c	 Terminated  	t3.micro
☒	First-server	i-0735c52dd3b29a35e	 Terminated  	t3.micro

3). Create one **EC2 instance** with **httpd installed** using a Terraform script.

- Create main.tf using user-data script

```

main.tf > resource "aws_instance" "webserver"
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "webserver" {
  ami           = "ami-0521cb2d60cfbb1a6"
  instance_type = "t3.micro"
  key_name      = "aws"

  user_data = <<-EOF
  #!/bin/bash
  yum update -y
  yum install httpd -y
  systemctl start httpd
  systemctl enable httpd
  echo "<h1>Apache Installed Using Terraform</h1>" > /var/www/html/index.html
  EOF

  tags = {
    Name = "Terraform-HTTPD-Server"
  }
}

```

- Now do terraform init

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.50.0...
- Installed hashicorp/aws v6.50.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

- Now Terraform plan and apply

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.webserver will be created
+ resource "aws_instance" "webserver" {
  + ami              = "ami-0521cb2d60cfbb1a6"
  + arn              = (known after apply)
  + associate_public_ip_address = (known after apply)
  + instance_type    = "t3.micro"
  + key_name          = (known after apply)
  + monitoring        = false
  + placement_group   = (known after apply)
  + subnet_id         = "subnet-0a123456"
  + tags              = {}
  + vpc_security_group_ids = ["sg-0a123456"]
}

Enter a value: yes

aws_instance.webserver: Creating...
aws_instance.webserver: Still creating... [00m10s elapsed]
aws_instance.webserver: Creation complete after 17s [id=i-075589ccc60abb25b]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

- Now check in AWS instance new instance created

Instances (1)

Info

Last updated 4 minutes ago

🔄

Connect

Instance state ▼

Saved filter sets

Running ▼

🔍

Find Instance by attribute or tag (case-sensitive)

Instance state = running

✕

Clear filters

▼

☐	Name ✎ ▼	Instance ID	Instance state ▼	Instance type ▼	Status check	Availability
☐	Terraform-HT...	i-075589ccc60abb25b	🟢 Running 🔍 🔍	t3.micro	🕒 Initializing	us-east-1a

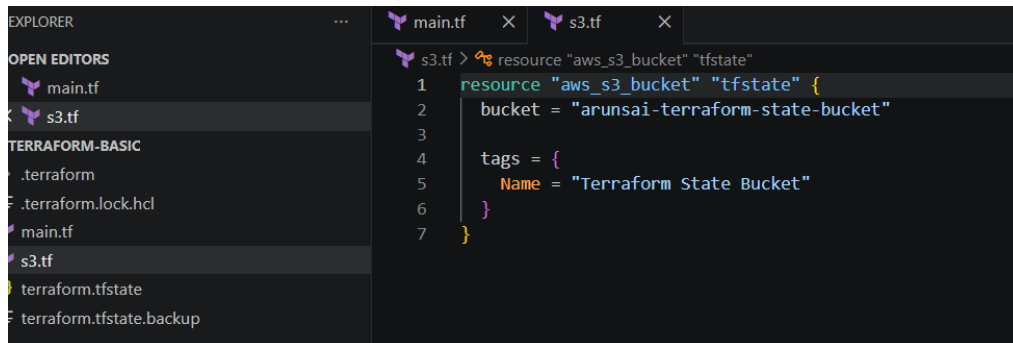
- Now check in browser with public Ip of instance



Apache Installed Using Terraform

4). Set up **S3 as backend** for task 3.

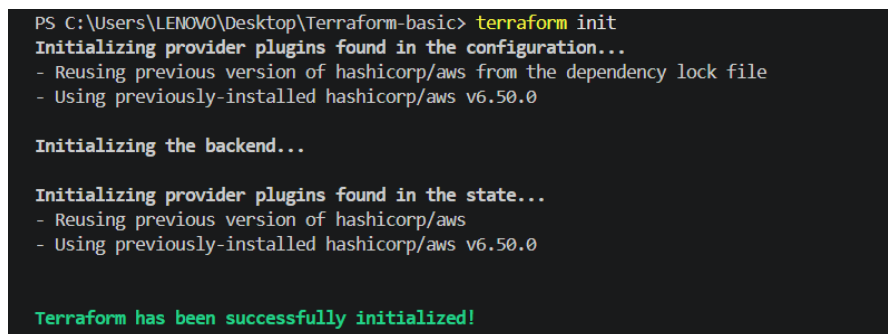
- Create s3.tf file and write a script



```
EXPLORER
OPEN EDITORS
main.tf
s3.tf
TERRAFORM-BASIC
.terraform
.terraform.lock.hcl
main.tf
s3.tf
terraform.tfstate
terraform.tfstate.backup

s3.tf
1 resource "aws_s3_bucket" "tfstate"
2   bucket = "arunsai-terraform-state-bucket"
3
4   tags = {
5     Name = "Terraform State Bucket"
6   }
7 }
```

- Then do terraform init



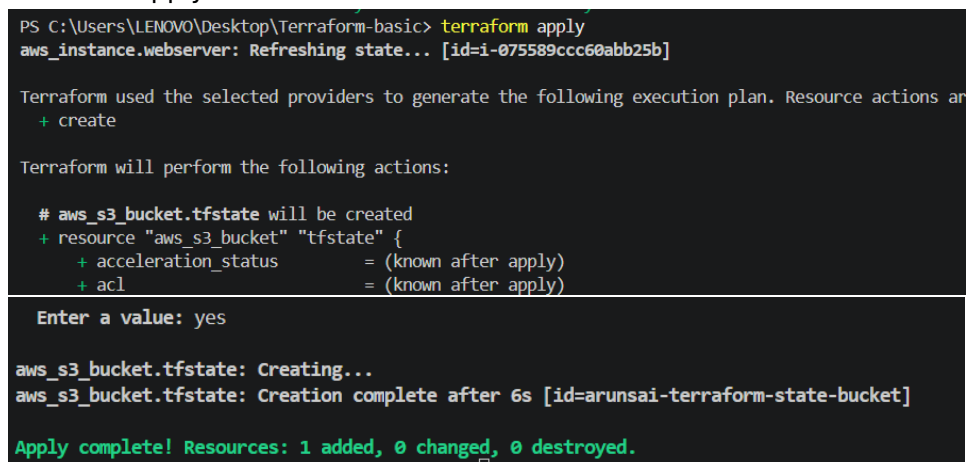
```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.50.0

Initializing the backend...

Initializing provider plugins found in the state...
- Reusing previous version of hashicorp/aws
- Using previously-installed hashicorp/aws v6.50.0

Terraform has been successfully initialized!
```

- Terraform apply



```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
aws_instance.webserver: Refreshing state... [id=i-075589ccc60abb25b]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

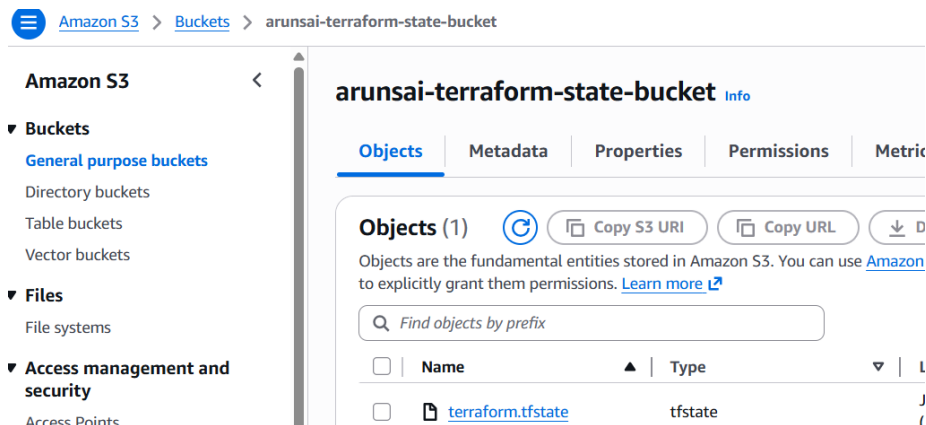
# aws_s3_bucket.tfstate will be created
+ resource "aws_s3_bucket" "tfstate" {
+   acceleration_status = (known after apply)
+   acl                 = (known after apply)

Enter a value: yes

aws_s3_bucket.tfstate: Creating...
aws_s3_bucket.tfstate: Creation complete after 6s [id=arunsai-terraform-state-bucket]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

- Now check in S3 bucket created or not



5). Set up **DynamoDB locking** for task 3.

- Create DynamoDB file and written script

```
dynamodb.tf > resource "aws_dynamodb_table" "terraform_lock"
1  resource "aws_dynamodb_table" "terraform_lock" {
2      name           = "terraform-lock"
3      billing_mode    = "PAY_PER_REQUEST"
4      hash_key        = "LockID"
5
6      attribute {
7          name = "LockID"
8          type = "S"
9      }
10 }
```

- And also written backend.tf file for terraform .tf state

```
main.tf  X  dynamodb.tf  backend.tf  X
backend.tf > terraform
1  terraform {
2      backend "s3" {
3          bucket = "arunsai-terraform-state-bucket"
4          key     = "terraform.tfstate"
5          region  = "us-east-1"
6      }
7  }
```

- Now do terraform init -reconfigure and check DynamoDB also

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform init -reconfigure
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.50.0

Initializing the backend...

Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.
```

- Now do terraform plan

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform plan
aws_s3_bucket.tfstate: Refreshing state... [id=arunsai-terraform-state-bucket]
aws_instance.webserver: Refreshing state... [id=i-075589ccc60abb25b]

Terraform used the selected providers to generate the following execution plan. Resource actions
+ create

Terraform will perform the following actions:

# aws_dynamodb_table.terraform_lock will be created
```

- Now do terraform apply

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
aws_s3_bucket.tfstate: Refreshing state... [id=arunsai-terraform-state-bucket]
aws_instance.webserver: Refreshing state... [id=i-075589ccc60abb25b]

Terraform used the selected providers to generate the following execution plan. Resources to be
+ create

Terraform will perform the following actions:
```

- Now check in DynamoDB Table is created

DynamoDB

Tables

Dashboard

Tables

Explore items

PartiQL editor

Backups

Exports to S3

Imports from S3

Tables (1) Info

June 14, 2026, 15:22 (UTC+5:30) Last updated

Find tables

Filter by tagAny tag key

Filter by tag valueAny tag value

	Name	Status	Partition key	Sort key	Indexes	Replication Regions
☐	terraform-lock	Active	LockID (S)	-	0	0